

CHAPTER 7

算術運算

Mathilog

本章主要介紹：數字系統，二進位數值系統 (Binary Numbering System) 中『無號數整數』、『有號數整數』、『無號數小數』和『有號數小數』的優缺點、可以表示的資料範圍，以及相對應的四則運算：『加法』、『減法』、『乘法』以及『除法』運算。

7.1 『數字系統』基本介紹

數字系統 (Numbering System) 就是表示數字的一種方法，我們最常使用的是十進位，又稱為基數十系統 (decimal)，因為它一共使用十個符號：0、1、2、3、4、5、6、7、8、9 來表示所有可能的數字。除了十進位以外，我們偶爾也會使用 16 進位，例如一台斤於 16 台兩、12 進制如 1 年 12 個月…等等。

日常生活中我們大都採用十進位來表示數字，但是電腦是由電子元件組成，所以其內部所有的資料 (包括：數值、字元、聲音、影像…等等) 都是以二進位方式存取的。

數值資料 (Numeric Data) 分成“整數”與“實數”，並且有“正數”與“負數”的分別。

整數有三種表示法，但這三種表示法都有範圍的限制，分別是符號大小 (Sign-magnitude) 表示法、1 的補數 (1's complement) 表示法與 2 的補數 (2's complement) 表示法。

表示法	正數範圍	負數範圍
符號大小	$0 \sim (2^{N-1} - 1)$	$-(2^{N-1} - 1) \sim 0$
1 的補數	$0 \sim (2^{N-1} - 1)$	$-(2^{N-1} - 1) \sim 0$
2 的補數	$0 \sim (2^{N-1} - 1)$	$-(2^{N-1}) \sim 0$

(一) 符號大小 (Sign-magnitude) 表示法

Sign-magnitude 表示法，最大正數為 $127(01111111)_2$ ，而最小的負數為 $-127(11111111)_2$ ；而 0 有兩種表示法，分別為：正 0 $(00000000)_2$ 與負 0 $(10000000)_2$ 。

使用 Sign-magnitude 表示法表示一個整數，最左邊的位元表示正負號（‘1’表示負數，‘0’表示正數），其餘的位元表示該數的大小。

Sign-magnitude 表示法的兩大缺點：有兩個 0（正 0 與負 0），不容易用邏輯電路設計加／減法器。

(二) 1 的補數 (1's complement) 表示法

最大正數為 $127(01111111)_2$ ，最小的負數為 $-127(10000000)_2$ ；而 0 有兩種表示法，分別為：正 0 $(00000000)_2$ 與負 0 $(11111111)_2$ 。

使用 1 的補數表示法表示一個整數和使用 Sign-magnitude 在“正”“負”號表示法以及“正數”的表示法是完全相同的，但“負數”的表示方法是不同的，必須將某正數表示法的所有位元：如果是‘0’則把它變成‘1’，如果是‘1’則要把它變成‘0’，得到該正數相對的負數值。

1 的補數表示法的兩大缺點：有兩個 0（正 0 與負 0），算術運算比 2 的補數表示法沒有效率。

(三) 2 的補數 (2's complement) 表示法

最大正數為 $127(01111111)_2$ ，而最小的負數為 $-128(10000000)_2$ ；而 0 只有一種表示法： $0(00000000)_2$ 。

使用 2 的補數表示法表示一個整數和使用 1 的補數表示法在“正”“負”號表示法以及“正數”的表示法是完全相同的，但“負數”的表示方法是不同

的，必須將某正數表示法的所有位元：如果是 '0' 則把它變成 '1'，如果是 '1' 則要把它變成 '0'，得到該正數相對的負數值，最後還要再加上 '1'，也就是說 $2's = 1's + 1$ 。

另外一種方式：先將正數表示出來之後從右邊掃描到左邊，如果是 '0'，則照抄下來，一直碰到第一個 '1' 仍然照抄，但是從這個 '1' 以後的左邊每個位元如果是 '1'，則將 '1' 改成 '0'；如果是 '0'，則將 '0' 改成 '1'，最後的結果就是 2 的補數表示法。



7.2 『乘法』的基本觀念

乘法運算中最基本的觀念為：某個數左移 1 個位元，就等於該數乘以 2。

其他不是 2 的幕次乘法，就得使用乘法電路才能辦到。



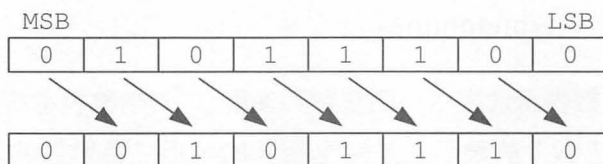
7.3 『除法』的基本觀念

除法運算中最基本的觀念為：某個數右移 1 個位元，就近似於或等於該數除以 2。

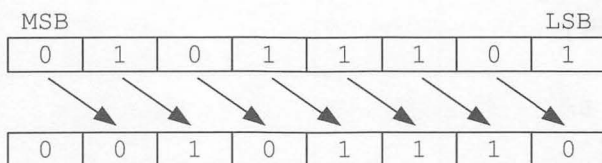
為什麼說是近似於該數除以 2？

舉個例子：

$A = 92$ ， A 除以 2 = 46，而 A 右移 1 個位元也等於 46。



$A=93$ ， A 除以 $2=46.5$ ，而 A 右移 1 個位元卻等於 46 (並不等於 46.5)。



所以求近似於或等於 2 的冪次除法運算，就可以用某個數右移的方法來達成。

其他不是 2 的冪次除法，就得使用除法電路才能辦到。



7.4 『無號數整數』的運算

無號數整數 (Unsigned Integer)，它只能做大於或等於 0 的整數加減乘除的運算。

■ 優點：容易運算，或者說是設計。

■ 缺點：無法表示“負”的數值。

假設 A 有 N 個位元，那麼 A 可以表示的資料範圍： $0 \sim 2^N - 1$ 。

例如 A 有 10 個位元，那麼 A 可以表示的資料範圍： $0 \sim 2^{10} - 1$ ，也就是 $0 \sim 1023$ 。

位置	9	8	7	6	5	4	3	2	1	0
二進制值	1	1	1	1	1	1	1	1	1	1
十進制值	512	256	128	64	32	16	8	4	2	1

7.4.1 無號數整數的『加法運算』

【Verilog 程式碼】

```
reg [ 9:0] A, B;
reg [10:0] C;
wire [10:0] C;

always @( A or B )
begin
    C = A + B;
end

assign D = A + B;
```

以 Verilog 的例子來看：A 和 B 和為 10 個 Bits，C 為 11 個 Bits。

A 和 B 為 unsigned 10 bits，可表示最大值為 1023，二進制表示法為 11 1111 1111；最小值為 0，二進制表示法為 00 0000 0000。

C 為 unsigned 11 bits，可表示最大值為 2047，二進制表示法為 111 1111 1111；最小值為 0，二進制表示法為 000 0000 0000。

A	最大值 1023 11 1111 1111	最大值 1023 11 1111 1111	最小值 0 00 0000 0000	最小值 0 00 0000 0000
B	最大值 1023 11 1111 1111	最小值 0 00 0000 0000	最大值 1023 11 1111 1111	最小值 0 00 0000 0000
C	2046 111 1111 1110	1023 011 1111 1111	1023 011 1111 1111	0 000 0000 0000

7.4.2 無號數整數的『減法運算』

【Verilog 程式碼】

```
reg [ 9:0] A, B, C;
wire [ 9:0] D;
```

```

always @( A or B )
begin
    C = A - B;
end

assign D = A - B;

```

以 Verilog 的例子來看：A 和 B 和為 10 個 Bits，C 為 10 個 Bits。

C 為 unsigned 10 bits，可表示最大值為 1023，二進制表示法為 11 1111 1111；最小值為 0，二進制表示法為 00 0000 0000。

A	最大值 1023 11 1111 1111	最大值 1023 11 1111 1111	最小值 0 00 0000 0000	最小值 0 00 0000 0000
B	最大值 1023 11 1111 1111	最小值 0 00 0000 0000	最大值 1023 11 1111 1111	最小值 0 00 0000 0000
C	0 00 0000 0000	1023 11 1111 1111	-1023 (無法表示) 00 0000 0001	0 00 0000 0000

注意 NOTICE

A 必須大於或等於 B，如果 B 比 A 大，那麼 A-B 的結果為負的，使用無號數的運算是無法表示負值的。所以上面的例子中，0-1023 應為 -1023，但 C 的值卻為 1。解決這個問題的方法之一就是改用接下來會提到的：有號數整數的『減法運算』。

減法要怎麼做呢？減法用(補數)加法來完成。

令 B's 為 B 的 2's 補數 (2's 補數 = 1's 補數 + 1)

$$A - B \Rightarrow A + (-B) \rightarrow A + B's$$

如果 A + B's 有進位，則運算完的值捨去進位 $R = A - B$

如果 A + B's 沒進位，則運算完的值 $R = A - B$

若最高位元 = 0，表 R 是正的， $A + B's = +R$

若最高位元 = 1，表 R 是負的，R 再取 2's 補數， $A + B's = -R$

7.4.3 無號數整數的『乘法運算』

A 乘以 2 以及 A 乘以 4。

【Verilog 程式碼】

```
reg [ 9:0] A;
reg [10:0] r1_A_Sh1_1;
reg [11:0] r2_A_Sh1_2;
wire [10:0] w1_A_Sh1_1;
wire [11:0] w2_A_Sh1_2;

always @( A )
begin
    r1_A_Sh1_1 = {A, 1'b0}; // A * 2
    r2_A_Sh1_2 = A << 2; // A * 4
end

assign w1_A_Sh1_1 = {A, 1'b0}; // A * 2
assign w2_A_Sh1_2 = A << 2; // A * 4
```

其他不是 2 的冪次乘法運算。

【Verilog 程式碼】

```
reg [ 9:0] A, B;
reg [19:0] C;
wire [19:0] D;

always @( A or B )
begin
    C = A * B;
end

assign D = A * B;
```

以 Verilog 的例子來看：A 和 B 和為 10 個 Bits，C 為 20 個 Bits。

C 為 unsigned 20 bits，可表示最大值為 1048575，二進制表示法為 1111 1111 1111 1111 1111；最小值為 0，二進制表示法為 0000 0000 0000 0000 0000。

A	最大值 1023 11 1111 1111	最大值 1023 11 1111 1111	最小值 0 00 0000 0000	最小值 0 00 0000 0000
B	最大值 1023 11 1111 1111	最小值 0 00 0000 0000	最大值 1023 11 1111 1111	最小值 0 00 0000 0000
C	1046529 1111 1111 1000 0000 0001	0 0000 0000 0000 0000 0000	0 0000 0000 0000 0000 0000	0 0000 0000 0000 0000 0000

7.4.4 無號數整數的『除法運算』

A 除以 2 以及 A 除以 4。

【Verilog 程式碼】

```
reg [ 9:0] A;
reg [ 9:0] r1_A_Shr_1, r2_A_Shr_2;
wire [ 9:0] w1_A_Shr_1, w2_A_Shr_2;

always @( A )
begin
    r1_A_Shr_1 = {1'b0, A[9:1]}; // A 除以 2
    r2_A_Shr_2 = A >> 2; // A 除以 4
end

assign w1_A_Shr_1 = {1'b0, A[9:1]}; // A 除以 2
assign w2_A_Shr_2 = A >> 2; // A 除以 4
```

其他不是 2 的冪次除法運算。

【Verilog 程式碼】

```
reg [ 9:0] A, B, C;
wire [ 9:0] D;

always @( A or B )
begin
    C = A / B;
end

assign D = A / B;
```

7.4.5 如何計算 1.5 倍的 A、3 倍的 A 呢？

動動腦時間 如何計算 1.5 倍的 A 呢？又要如何計算 3 倍的 A 呢？

1.5 倍的 A 相等或近似於 A 再加上 A 右移 1 個位元，而 3 倍的 A 等於 A 左移 1 個位元再加上 A。

【Verilog 程式碼】

```
reg [ 9:0] A;
wire [ 9:0] A_Shr_1;
wire [ 10:0] A_Mul_1d5;
wire [ 10:0] A_Sh1_1;
wire [ 11:0] A_Mul_3;

assign A_Shr_1 = {1'b0, A[9:1]}; // A 除以 2
assign A_Mul_1d5 = A + A_Shr_1; // 1.5 倍的 A
assign A_Sh1_1 = {A, 1'b0}; // A * 2
assign A_Mul_3 = A + A_Sh1_1; // 3 倍的 A
```

7.4.6 如何計算 A 除以 9？

在 4.1.2 不能用於電路合成的「運算子」裡頭有提到：除法運算子(/, division)只能用來模擬，不過並不能由合成軟體來做電路合成的。

想想看，當您需要將某個數(例如：A)除以 9，那麼該怎麼設計呢？因為某數除以 9 最接近於某數除以 8(右移 3 個位元)，以及減去誤差值，那麼 $A/9$ 近似於 $(A - (A \text{ 右移 } 3 \text{ 個位元}))$ ，之後再右移 3 個位元。您還可以再想想看其他計算某數除以 9 的近似值或某數除以 N 的近似值的方法。

【Verilog 程式碼】

```
input [7:0] a;
output [7:0] out;

wire [7:0] b;
```

```

wire    [7:0] a_sub_b;
wire    [7:0] out;

assign b = {3'b0, a[7:3]};           // a >> 3
assign a_sub_b = a - b;              // a - (a >> 3)
assign out = {3'b0, a_sub_b[7:3]};   // (a - (a >> 3)) >> 3

```

7.4.7 四捨五入的方式

四捨五入的方式就是將該數值加上 0.5，再右移小數點的位數。

例 1 小數點有 2 位數，先加 $(0.10)_2$ ，也就是加 2 進制的 2

$(101.11)_2 + (0.10)_2 = (110.01)_2 \rightarrow$ 再右移 2 位得到 $(110)_2$

進位

$(101.01)_2 + (0.10)_2 = (101.11)_2 \rightarrow$ 再右移 2 位得到 $(101)_2$

沒有進位

例 2 小數點有 3 位數，先加 $(0.100)_2$ ，也就是加 2 進制的 4

$(101.101)_2 + (0.100)_2 = (110.001)_2 \rightarrow$ 再右移 3 位得到 $(110)_2$

進位

$(101.001)_2 + (0.100)_2 = (101.101)_2 \rightarrow$ 再右移 3 位得到 $(101)_2$

沒有進位

例 3 小數點有 4 位數，先加 $(0.1000)_2$ ，也就是加 2 進制的 8

$(101.1001)_2 + (0.1000)_2 = (110.0001)_2 \rightarrow$ 再右移 4 位得到 $(110)_2$

進位

$(101.0001)_2 + (0.1000)_2 = (101.1001)_2 \rightarrow$ 再右移 4 位得到 $(101)_2$

沒有進位



7.5 『有號數整數』的運算

有號數整數 (Signed Integer)，也稱為二的補數整數 (2's Complement Integer)。

- 優點：容易運算，可以表示“正”與“負”的數值。
- 缺點：必須多用一個位元 (通常是用最高位元，也就是 MSB，Most Significant Bit，0：正、1：負) 來表示“正”與“負”的數值。

假設 A 有 N 個位元，那麼 A 可以表示的資料範圍： $-2^{N-1} \sim 2^{N-1} - 1$ 。

例如 A 有 10 個位元，那麼 A 可以表示的資料範圍： $-2^{10-1} \sim 2^{10-1} - 1$ ，也就是 $-512 \sim 511$ 。

7.5.1 有號數整數的『加法運算』

【Verilog 程式碼】

```
reg signed [ 9:0] A, B;  
reg signed [10:0] C;  
wire signed [10:0] D;  
  
always @( A or B )  
begin  
    C = A + B;  
end  
  
assign D = A + B;
```

Verilog-2001 增加了 reg 及 wire 這二種資料型態可以宣告成 signed (有號數)，更多說明請看本書 13.7 Signed Arithmetic Extension 這個章節。

以 Verilog 的例子來看：A 和 B 和為 10 個 Bits，C 為 11 個 Bits。

A 和 B 爲 signed 10 bits，可表示最大值爲 511，二進制表示法爲 01 1111 1111；最小值爲 -512，二進制表示法爲 10 0000 0000。

C 爲 signed 11 bits，可表示最大值爲 1023，二進制表示法爲 011 1111 1111；最小值爲 -1024，二進制表示法爲 100 0000 0000。

A	最大值 511 01 1111 1111	最大值 511 01 1111 1111	最小值 -512 10 0000 0000	最小值 -512 10 0000 0000
B	最大值 511 01 1111 1111	最小值 -512 10 0000 0000	最大值 511 01 1111 1111	最小值 -512 10 0000 0000
C	1022 011 1111 1110	-1 111 1111 1111	-1 111 1111 1111	-1024 100 0000 0000

7.5.2 有號數整數的『減法運算』

【Verilog 程式碼】

```
reg signed [ 9:0] A, B;
reg signed [10:0] C;
wire signed [10:0] D;

always @( A or B )
begin
    C = A - B;
end

assign D = A - B;
```

A	最大值 511 01 1111 1111	最大值 511 01 1111 1111	最小值 -512 10 0000 0000	最小值 -512 10 0000 0000
B	最大值 511 01 1111 1111	最小值 -512 0000000000	最大值 511 01 1111 1111	最小值 -512 10 0000 0000
C	0 000 0000 0000	1023 011 1111 1111	-1023 100 0000 0001	0 000 0000 0000

7.5.3 有號數整數的『乘法運算』

A 乘以 2 以及 A 乘以 4。

【Verilog 程式碼】

```
reg signed [ 9:0] A;
reg signed [10:0] r1_A_Sh1_1;
reg signed [11:0] r2_A_Sh1_2;
wire signed [10:0] w1_A_Sh1_1;
wire signed [11:0] w2_A_Sh1_2;

always @( A )
begin
    r1_A_Sh1_1 = $signed(A) * 2; // signed(A) * 2
    r2_A_Sh1_2 = A <<< 2; // signed(A) * 4
end

assign w1_A_Sh1_1 = $signed(A) * 2; // signed(A) * 2
assign w2_A_Sh1_2 = A <<< 2; // signed(A) * 4
```

其他不是 2 的冪次乘法運算。

【Verilog 程式碼】

```
reg signed [ 9:0] A, B;
reg signed [19:0] C;
wire signed [19:0] D;

always @( A or B )
begin
    C = A * B;
end

assign D = A * B;
```

C 為 signed 20 bits，可表示最大值為 524287，二進制表示法為 111 1111 1111 1111；最小值為 -524288，二進制表示法為 1000 0000 0000 0000 0000。

A	最大值 511 01 1111 1111	最大值 511 01 1111 1111	最小值 -512 10 0000 0000	最小值 -512 10 0000 0000
B	最大值 511 01 1111 1111	最小值 -512 10 0000 0000	最大值 511 01 1111 1111	最小值 -512 10 0000 0000
C	261121 0011 1111 1100 0000 0001	-261632 1100 0000 0010 0000 0000	-261632 1100 0000 0010 0000 0000	262144 0100 0000 0000 0000 0000

7.5.4 有號數整數的『除法運算』

A 除以 2 以及 A 除以 4。

【Verilog 程式碼】

```
reg signed [ 9:0] A;
reg signed [ 9:0] r1_A_Shr_1, r2_A_Shr_2;
wire signed [ 9:0] w1_A_Shr_1, w2_A_Shr_2;

always @( A )
begin
    r1_A_Shr_1 = $signed(A) / 2; // signed(A) 除以 2
    r2_A_Shr_2 = A >>> 2; // signed(A) 除以 4
end

assign w1_A_Shr_1 = $signed(A) / 2; // signed(A) 除以 2
assign w2_A_Shr_2 = A >>> 2; // signed(A) 除以 4
```

其他不是 2 的冪次除法運算。

【Verilog 程式碼】

```
reg signed [ 9:0] A, B, C;
wire signed [ 9:0] D;

always @( A or B )
begin
    C = A / B;
end

assign D = A / B;
```




7.6 『無號數小數』的運算

無號數小數 (Unsigned Fractional)，也稱為二的補數整數 (2's Complement Integer)。

■ 優點：可以表示“正”的整數與小數。

■ 缺點：無法表示“負”的整數與小數。

假設 A 的整數部份有 N 個位元，小數部份有 M 個位元，那麼 A 可以表示的資料範圍： $0 \sim (2^N - 2^{-M})$ 。

例如 A 的整數部份有 5 個位元，小數部份有 6 個位元，那麼 A 可以表示的資料範圍： $0 \sim (2^5 - 2^{-6})$ ，也就是 $0 \sim 2^5 - (1 / 2^6)$ 。

位置	4	3	2	1	0	-1	-2	-3	-4	-5
二進制值	1	1	1	1	1	1	1	1	1	1
十進制值	16	8	4	2	1	0.5	0.25	0.125	0.0625	0.03125

其他的浮點數表示：

① 浮點式表示法

(Floating Point Representation，又稱為小數表示法)

格式：符號位元 (Sign) 指數 (Exponent) 假數 (Mantissa)。

② IEEE (Institute of Electrical and Electronic Engineers，國際電機電子工程協會) 浮點數規格

單精確度 (32 Bits) 及倍精確度 (64 Bits)。

這二種方法的優點會比我們底下使用整數模擬小數來作小數運算有較高的精確度，但它們的缺點均為實作不易，而且電路面積十分地大。

7.6.1 無號數小數的『加法運算』

用二進制來表示無號數小數『加法運算』。

【Verilog 程式碼】

```
reg [ 9:0] A, B;
reg [10:0] C;
wire [10:0] D;

always @( A or B )
begin
    C = A + B;
end

assign D = A + B;
```

以 Verilog 的例子來看：A 和 B 和為 10 個 Bits，C 為 11 個 Bits。

假設 A 和 B 的整數部份各有 5 個位元，小數部份各有 5 個位元，可表示最大值為 31.96875，二進制表示法為 11 1111 1111；最小值為 0.0，二進制表示法為 00 0000 0000。

假設 C 的整數部份有 6 個位元，小數部份有 5 個位元，可表示最大值為 63.96875，二進制表示法為 111 1111 1111；最小值為 0.0，二進制表示法為 000 0000 0000。

A	最大值 31.96875 11 1111 1111	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000	最小值 0.0 00 0000 0000
B	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000
C	63.9375 111 1111 1110	31.96875 011 1111 1111	31.96875 011 1111 1111	0.0 000 0000 0000
C 的 整數部份	63 11 1111	31 01 1111	31 01 1111	0 00 0000

7.6.2 無號數小數的『減法運算』

用二進制來表示無號數小數『減法運算』。

【Verilog 程式碼】

```
reg [ 9:0] A, B, C;
wire [ 9:0] D;

always @( A or B )
begin
    C = A - B;
end

assign D = A - B;
```

A	最大值 31.96875 11 1111 1111	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000	最小值 0.0 00 0000 0000
B	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000
C	0.0 00 0000 0000	31.96875 11 1111 1111	-31.96875 (無法表示) 00 0000 0001	0.0 00 0000 0000
C 的 整數部份	0 00 0000	31 01 1111	31 01 1111 (無法表示 -31)	0 00 0000

7.6.3 無號數小數的『乘法運算』

用二進制來表示無號數小數，A 乘以 2 以及 A 乘以 4。

【Verilog 程式碼】

```
reg [ 9:0] A;
reg [10:0] r1_A_Sh1_1;
reg [11:0] r2_A_Sh1_2;
```

```

wire [10:0] w1_A_Sh1_1;
wire [11:0] w2_A_Sh1_2;

always @( A )
begin
    r1_A_Sh1_1 = {A, 1'b0}; // A * 2
    r2_A_Sh1_2 = A << 2; // A * 4
end

assign w1_A_Sh1_1 = {A, 1'b0}; // A * 2
assign w2_A_Sh1_2 = A << 2; // A * 4

```

其他不是 2 的幕次乘法運算。

【Verilog 程式碼】

```

reg [ 9:0] A, B;
reg [19:0] C;
wire [19:0] D;

always @( A or B )
begin
    C = A * B;
end

assign D = A * B;

```

以 Verilog 的例子來看：A 和 B 和為 10 個 Bits，C 為 20 個 Bits。

A	最大值 31.96875 11 1111 1111	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000	最小值 0.0 00 0000 0000
B	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000	最大值 31.96875 11 1111 1111	最小值 0.0 00 0000 0000
C	1022.0009765625 1111 1111 10 00 0000 0001	0.0 0000 0000 00 00 0000 0000	0.0 0000 0000 00 00 0000 0000	0.0 0000 0000 00 00 0000 0000
C 的 整數部份	1022 11 1111 1110	0 00 0000 0000	0 00 0000 0000	0 00 0000 0000

C 的整數部份有 10 個位元，小數部份有 10 個位元，可表示最大值為 1023.9990234375，二進制表示法為 11 1111 1111；最小值為 0.0，二進制表示法為 00 0000 0000。

7.6.4 無號數小數的『除法運算』

用二進制來表示無號數小數，A 除以 2 以及 A 除以 4。

【Verilog 程式碼】

```
reg [ 9:0] A;
reg [ 9:0] r1_A_Shr_1, r2_A_Shr_2;
wire [ 9:0] w1_A_Shr_1, w2_A_Shr_2;

always @( A )
begin
    r1_A_Shr_1 = {1'b0, A[9:1]}; // A 除以 2
    r2_A_Shr_2 = A >> 2; // A 除以 4
end

assign w1_A_Shr_1 = {1'b0, A[9:1]}; // A 除以 2
assign w2_A_Shr_2 = A >> 2; // A 除以 4
```

其他不是 2 的冪次除法運算。

【Verilog 程式碼】

```
reg [ 9:0] A, B, C;
wire [ 9:0] D;

always @( A or B )
begin
    C = A / B;
end

assign D = A / B;
```



7.7 『有號數小數』的運算

有號數整數小數 (Signed Fractional)，也稱為二的補數小數 (2's Complement Signed Fractional)。

- **優點：**容易運算，可以表示“正”與“負”的數值。
- **缺點：**必須多用一個位元 (通常是用最高位元，也就是 MSB，Most Significant Bit，0：正、1：負) 來表示“正”與“負”的數值。

例如 A 一共有 11 個位元，其中 A 的符號位元需 1 個位元，A 的整數部份有 5 個位元，小數部份有 6 個位元，那麼 A 可以表示的資料範圍： $-2^5 \sim (2^5 - 2^{-6})$ ，也就是 $-2^5 \sim 2^5 - (1 / 2^6)$ 。

7.7.1 有號數小數的『加法運算』

用二進制來表示有號數小數『加法運算』。

【Verilog 程式碼】

```
reg signed [10:0] A, B;
reg signed [11:0] C;
wire signed [11:0] D;

always @( A or B )
begin
    C = A + B;
end

assign D = A + B;
```

以 Verilog 的例子來看：A 和 B 和為 11 個 Bits，C 為 12 個 Bits。

A 和 B 的整數部份各有 5 個位元，小數部份各有 5 個位元，可表示最大值為 31.96875，二進制表示法為 011 1111 1111；最小值為 -32.0，二進制

表示法為 100 0000 0000。

C 的整數部份有 6 個位元，小數部份有 5 個位元，可表示最大值為 63.96875，二進制表示法為 0111 1111 1111；最小值為 -64.0，二進制表示法為 1000 0000 0000。

A	最大值 31.96875 011 1111 1111	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000	最小值 -32.0 100 0000 0000
B	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000
C	63.875 0111 1111 1110	-1 1111 1111 1111	-1 1111 1111 1111	-64 1000 0000 0000
C 的 整數加上 小數進位	63 011 1111	-1 111 1111	-1 111 1111	-64 100 0000

7.7.2 有號數小數的『減法運算』

用二進制來表示有號數小數『減法運算』。

【Verilog 程式碼】

```
reg signed [10:0] A, B, C;
reg signed [10:0] C;
wire signed [10:0] D;

always @( A or B )
begin
    C = A - B;
end

assign D = A + B;
```

A	最大值 31.96875 011 1111 1111	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000	最小值 -32.0 100 0000 0000
B	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000
C	0.0 000 0000 0000	63.96875 111 1111 1111	-63.96875 000 0000 0001	0.0 000 0000 0000
C 的 整數加上 小數進位	0 000 0000	63 011 1111	-64 100 0000	0 000 0000

7.7.3 有號數小數的『乘法運算』

用二進制來表示有號數小數，A 乘以 2 以及 A 乘以 4。

【Verilog 程式碼】

```
reg signed [ 9:0] A;
reg signed [10:0] r1_A_Sh1_1;
reg signed [11:0] r2_A_Sh1_2;
wire signed [10:0] w1_A_Sh1_1;
wire signed [11:0] w2_A_Sh1_2;

always @( A )
begin
    r1_A_Sh1_1 = $signed(A) * 2; // signed(A) * 2
    r2_A_Sh1_2 = A << 2; // signed(A) * 4
end

assign w1_A_Sh1_1 = $signed(A) * 2; // signed(A) * 2
assign w2_A_Sh1_2 = A << 2; // signed(A) * 4
```

其他不是 2 的冪次乘法運算。

【Verilog 程式碼】

```
reg signed [ 9:0] A, B;
reg signed [19:0] C;
wire signed [19:0] D;

always @( A or B )
begin
    C = A * B;
end

assign D = A * B;
```

以 Verilog 的例子來看：A 和 B 和為 11 個 Bits，C 為 22 個 Bits。

C 一共有 22 個位元，其中 C 的符號位元需 1 個位元，C 的整數部份有 10 個位元，小數部份有 10 個位元，可表示最大值為 1023.9990234375，二進制表示法為 01 1111 1111 1111 1111 1111；最小值為 -1024.0，二進制表示法為 10 0000 0000 0000 0000 0000。

A	最大值 31.96875 011 1111 1111	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000	最小值 -32.0 100 0000 0000
B	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000	最大值 31.96875 011 1111 1111	最小值 -32.0 100 0000 0000
C	1022.0009765625 0011 1111 1110	-1023.0 1100 0000 0001	-1023.0 1100 0000 0001	1024.0 0100 0000 0000
C 的 整數加上 小數進位	1022 0011 1111 1110	-1023 1100 0000 0001	-1023 1100 0000 0001	1024 0100 0000 0000

7.7.4 有號數小數的『除法運算』

用二進制來表示有號數小數，A 除以 2 以及 A 除以 4。

【Verilog 程式碼】

```
reg signed [ 9:0] A;
reg signed [ 9:0] r1_A_Shr_1, r2_A_Shr_2;
wire signed [ 9:0] w1_A_Shr_1, w2_A_Shr_2;

always @( A )
begin
    r1_A_Shr_1 = $signed(A) / 2; // signed(A) 除以 2
    r2_A_Shr_2 = A >>> 2; // signed(A) 除以 4
end

assign w1_A_Shr_1 = $signed(A) / 2; // signed(A) 除以 2
assign w2_A_Shr_2 = A >>> 2; // signed(A) 除以 4
```

其他不是 2 的冪次除法運算。

【Verilog 程式碼】

```
reg signed [ 9:0] A, B, C;
wire signed [ 9:0] D;

always @( A or B )
begin
    C = A / B;
end

assign D = A / B;
```

本章練習

Question 1 :

請設計一個能將輸入的值做 $\times 7$ 運算的『無號數整數乘法』電路。

Question 2 :

請設計一個能將輸入的值做近似於 $/ 9$ 運算的『無號數整數除法』電路。

Question 3 :

四捨五入的方式就是將該數值加上 0.5，再右移小數點的位數，請設計並驗證相關的電路。

例 1：小數點有 2 位數、先加 $(0.10)_2$ ，也就是加 2 進制的 2

$$(101.11)_2 + (0.10)_2 = (110.01)_2 \rightarrow \text{再右移 2 位得到 } (110)_2$$

進位

$$(101.01)_2 + (0.10)_2 = (101.11)_2 \rightarrow \text{再右移 2 位得到 } (101)_2$$

沒有進位

例 2：小數點有 3 位數、先加 $(0.100)_2$ ，也就是加 2 進制的 4

$$(101.101)_2 + (0.100)_2 = (110.001)_2 \rightarrow \text{再右移 3 位得到 } (110)_2$$

進位

$$(101.001)_2 + (0.100)_2 = (101.101)_2 \rightarrow \text{再右移 3 位得到 } (101)_2$$

沒有進位

PS Google 可以幫您找到解答。